

UNIVERSITÉ
Département Réseaux Informatiques

Travail Pratique 3

Code de Hamming

**Correction Automatique d'Erreurs en Transmission
de Données**

Étudiant : Nathan ADOHO

Date : 5 février 2026

Année académique : 2025-2026

Objectif Pédagogique

Ce TP vise à approfondir la compréhension des codes de correction d'erreurs, en particulier le code de Hamming, qui constitue une classe fondamentale de codes linéaires utilisés dans les systèmes critiques nécessitant une détection et une correction automatiques d'erreurs.

Rapport TP3 - Code de Hamming

Nathan ADOHO

5 février 2026

Table des matières

1	Rappel du Sujet	2
1.1	Contexte Historique et Importance	2
1.2	Objectifs Principaux du TP	2
1.3	Enjeux Pratiques et Critiques	2
2	Analyse du Sujet	3
2.1	Contexte Théorique Approfondi	3
2.2	Historique et Évolution	3
2.3	Principes Mathématiques Fondamentaux	3
2.3.1	Structure du Code	3
2.3.2	Positionnement des Bits	3
2.3.3	Calcul des Bits de Parité	4
2.3.4	Vérification et Correction	4
2.4	Efficacité du Code	4
3	Choix Techniques Effectués	4
3.1	Architecture Générale	4
3.2	Détails d'Implémentation	5
3.2.1	Méthode d'Encodage - motToCodeAndHamming	5
3.2.2	Méthode de Vérification - verifHamming	6
3.3	Interface Utilisateur	7
4	Résultats et Tests	7
4.1	Cas de Test 1 : Encodage Simple	7
4.2	Cas de Test 2 : Vérification Sans Erreur	7
4.3	Cas de Test 3 : Détection et Correction d'Erreur	7
4.4	Validation Empirique	7
5	Conclusion	7
5.1	Apprentissages Clés	7
5.2	Difficultés Rencontrées	8
5.3	Limites du Programme	8
5.4	Perspectives d'Évolution	8
5.5	Conclusion Générale	8

1 Rappel du Sujet

Le travail pratique 3 portait sur l'implémentation et la compréhension du **Code de Hamming**, un code linéaire de correction d'erreurs fondamental en théorie de l'information et en communications numériques.

1.1 Contexte Historique et Importance

Le code de Hamming a révolutionné le domaine des télécommunications depuis son invention en 1950. Ses applications s'étendent à :

- Les transmissions spatiales (missions NASA, satellites)
- Les mémoires RAM (correction d'erreurs ECC)
- Les disques durs et SSD
- Les communications sans fil (3G, 4G, 5G)
- Les codes QR
- L'astronomie et le traitement du signal

En 2025, le code de Hamming reste un élément clé de la technologie ECC (Error Correcting Code) présente dans pratiquement tous les ordinateurs et appareils électroniques modernes.

1.2 Objectifs Principaux du TP

Les objectifs pédagogiques étaient :

1. Comprendre le fonctionnement mathématique du code de Hamming et sa théorie des bits de parité
2. Implémenter un encodeur transformant un message binaire en mot de code Hamming
3. Implémenter un décodeur capable de détecter et de corriger automatiquement les erreurs
4. Développer une interface graphique permettant de tester l'encodage et la correction
5. Valider empiriquement que l'algorithme corrige correctement une seule erreur par mot de code

1.3 Enjeux Pratiques et Critiques

Considérons les conséquences d'une erreur non corrigée :

- Dans les communications spatiales : perte définitive du signal
- Dans les mémoires RAM : corruption des données traitées
- Dans les bases de données : intégrité des données compromise
- Dans les dispositifs médicaux : risque pour la vie du patient
- Dans l'aviation : sécurité des passagers compromise

Le code de Hamming offre une solution élégante et efficace à ces défis critiques.

2 Analyse du Sujet

2.1 Contexte Théorique Approfondi

Le code de Hamming, inventé par Richard Hamming en 1950 aux Bell Laboratories, est un code linéaire de correction d'erreurs qui peut :

- **Détecter** jusqu'à 2 erreurs de bit simultanées
- **Corriger** automatiquement 1 erreur de bit

Contrairement au CRC (TP2) qui ne détecte que les erreurs, Hamming les corrige automatiquement, ce qui en fait un code **autocorrecteur**.

2.2 Historique et Évolution

- **1950** : Richard Hamming invente le code aux Bell Labs
- **1956** : Publication complète de la théorie
- **1960s-1970s** : Adoption dans les systèmes informatiques
- **1980s** : Intégration massive dans les mémoires RAM (ECC RAM)
- **1990s-2025** : Élément omniprésent dans tous les systèmes critiques

2.3 Principes Mathématiques Fondamentaux

2.3.1 Structure du Code

Pour encoder un message de m bits, il faut ajouter r bits de parité, où r est le nombre minimum satisfaisant :

$$2^r \geq m + r + 1$$

Exemples pratiques :

Bits de données (m)	Bits de parité (r)	Longueur totale	Code
1	2	3	[3,1]
2	2	4	[4,2]
3	3	6	[6,3]
4	3	7	[7,4]
8	4	12	[12,8]
16	5	21	[21,16]

2.3.2 Positionnement des Bits

Les bits de parité sont placés aux positions $2^0, 2^1, 2^2, \dots$ (positions 1, 2, 4, 8, 16, ...) dans le mot de code. Les bits de données occupent les autres positions.

Exemple pour un code [7,4] :

Position: 1 2 3 4 5 6 7
Rôle: P1 P2 D1 P3 D2 D3 D4
 ^ ^ ^

Bits de parité (puissances de 2)

2.3.3 Calcul des Bits de Parité

Chaque bit de parité à la position 2^i est calculé comme la parité XOR de tous les bits dont le i -ème bit est 1 dans leur représentation binaire de position.

Pour un code [7,4] :

- **Parité 1** : positions dont bit 0 = 1 : {1,3,5,7}
- **Parité 2** : positions dont bit 1 = 1 : {2,3,6,7}
- **Parité 4** : positions dont bit 2 = 1 : {4,5,6,7}

Mathématiquement :

$$P_1 = D_1 \oplus D_2 \oplus D_4 \quad (1)$$

$$P_2 = D_1 \oplus D_3 \oplus D_4 \quad (2)$$

$$P_3 = D_2 \oplus D_3 \oplus D_4 \quad (3)$$

où $\oplus$ représente l'opération XOR.

2.3.4 Vérification et Correction

À la réception, on calcule le **syndrome** en refaisant les calculs de parité. Le syndrome indique la position exacte du bit en erreur :

$$\text{syndrome} = \sum_{i=0}^{r-1} (\text{parité}_i \text{ incorrecte}) \times 2^i$$

- Si syndrome = 0 : aucune erreur
- Si syndrome $\neq$ 0 : le bit à la position 'syndrome' est en erreur et peut être corrigé automatiquement

2.4 Efficacité du Code

Le rendement du code de Hamming [n,k] est défini par :

$$\eta = \frac{k}{n} = \frac{m}{m+r}$$

Exemples :

- Code [7,4] : $\eta = 4/7 \approx 57.1\%$
- Code [15,11] : $\eta = 11/15 \approx 73.3\%$
- Code [31,26] : $\eta = 26/31 \approx 83.9\%$

Le rendement s'améliore avec la taille du code, ce qui rend Hamming particulièrement efficace pour les longs messages.

3 Choix Techniques Effectués

3.1 Architecture Générale

Le projet est structuré en deux classes principales :

1. **Hamming.java** : Implémentation des algorithmes d'encodage et de vérification
2. **Fenetre.java** : Interface graphique Swing pour tester les fonctionnalités

3.2 Détails d'Implémentation

3.2.1 Méthode d'Encodage - motToCodeAndHamming

La méthode `motToCodeAndHamming(String mot)` transforme un message binaire en code de Hamming :

Listing 1 – Méthode `motToCodeAndHamming` (Hamming.java)

```
5      int r;
6      int j = 0;
7      int k = 0;
8      int m = mot.length();
9      String[] motFinal;
10     String code = "";
11     // Calculer le nombre de bits de parité nécessaires
12     for (r = 0; ; r++) {
13         if (Math.pow(2, r) >= m + r + 1)
14             break;
15     }
16     // Placer les bits de données et les bits de parité
17     motFinal = new String[m + r];
18     for (int i = 1; i <= motFinal.length; i++) {
19         if (Math.pow(2, k) == i) {
20             motFinal[i - 1] = "0";
21             k++;
22         }
23         else {
24             motFinal[i - 1] = String.valueOf(mot.charAt(j));
25             j++;
26         }
27     }
28     // Calculer les bits de parité
29     for (int i = 0; i < r; i++) {
30         int posParite = (int) Math.pow(2, i);
31         int bitValeur = 0;
32
33         for (int a = 1; a <= motFinal.length; a++) {
34             if (((a >> i) & 1) == 1)
35                 if (a != posParite)
36                     bitValeur ^= Integer.parseInt(motFinal[a - 1]);
37         }
38         motFinal[posParite - 1] = String.valueOf(bitValeur);
39     }
40     // Construire le code adjoindre final
41     k = 0;
42     for (int i = 0; i < motFinal.length; i++) {
43         if (Math.pow(2, k) == i + 1) {
44             code += motFinal[i] ;
45             k++;
46         }
47     }
```

```

48
49         return new String[]{
50             String.join("", motFinal),

```

Processus :

1. **Calcul de r** : Détermine le nombre de bits de parité nécessaires via $2^r \geq m + r + 1$
2. **Insertion des bits** : Place les bits du message aux positions non-puissance de 2
3. **Calcul des parités** : Remplit les positions de parité en utilisant le XOR
4. **Extraction du code** : Construit le code de parité à ajouter

3.2.2 Méthode de Vérification - `verifHamming`

La méthode `verifHamming(String codeHamming)` vérifie et détecte les erreurs :

Listing 2 – Méthode `verifHamming` (Hamming.java)

```

52     };
53 }
54
55 protected static boolean verifHamming(String codeHamming) {
56     int syndrome = 0;
57     int r;
58     // Calculer le nombre de bits de parité
59     for (r = 0; ; r++) {
60         if (Math.pow(2, r) >= codeHamming.length() + 1)
61             break;
62     }
63     // Calculer le syndrome
64     for (int i = 0; i < r; i++) {
65         int posParite = (int) Math.pow(2, i);
66         int bitValeur = 0;
67
68         for (int a = 1; a <= codeHamming.length(); a++) {
69             if (((a >> i) & 1) == 1)
70                 if (a != posParite)
71                     bitValeur ^= Integer.parseInt(String.
72                         valueOf(codeHamming.charAt(a - 1)));
73         }
74         bitValeur ^= Integer.parseInt(String.valueOf(
75             codeHamming.charAt(posParite - 1)));
76         syndrome += bitValeur * posParite;
77     }

```

Processus :

1. Recalcule les parités à partir du code reçu
2. Calcule le syndrome (somme des positions des parités incorrectes)
3. Si syndrome = 0 : pas d'erreur
4. Si syndrome $\neq$ 0 : indique la position exacte du bit en erreur

Le syndrome est calculé en multipliant chaque parité incorrecte par sa position (puissance de 2 correspondante), ce qui donne directement la position du bit en erreur.

3.3 Interface Utilisateur

L'interface Swing permet :

- **Sélection du mode** : Calculer Code ou Vérifier Code via une ComboBox
- **Zone d'entrée** : accepte uniquement des bits (0 et 1)
- **Zone de sortie** : affiche le code Hamming ou le résultat de vérification
- **Boutons** : Exécuter, Effacer

4 Résultats et Tests

4.1 Cas de Test 1 : Encodage Simple

Entrée : Message = 1011 (4 bits de données)

Calcul :

- Nombre de bits de parité nécessaires : $r = 3$ (car $2^3 = 8 \geq 4 + 3 + 1$)
- Longueur du code final : 7 bits

Résultat attendu : Un code de 7 bits incluant les 4 bits de données et 3 bits de parité

4.2 Cas de Test 2 : Vérification Sans Erreur

Entrée : Code Hamming = résultat du test 1

Résultat attendu : Syndrome = 0, pas d'erreur détectée

4.3 Cas de Test 3 : Détection et Correction d'Erreur

Entrée : Le même code avec un bit inversé à la position 3

Processus :

1. Calcul du syndrome $\rightarrow$ indique position 3
2. Correction automatique du bit
3. Récupération du message original

Résultat : Le code détecte et corrige l'erreur automatiquement

4.4 Validation Empirique

L'implémentation a été validée sur :

- Messages de longueurs variables (4 bits à 16 bits et plus)
- Injection d'erreurs à différentes positions
- Vérification que le syndrome indique la position correcte

Tous les tests ont confirmé le bon fonctionnement de l'encodeur et du correcteur.

5 Conclusion

5.1 Apprentissages Clés

Ce TP a approfondi la compréhension de :

- La théorie de la correction d'erreurs et ses applications pratiques

- L'arithmétique binaire et le XOR pour les calculs de parité
- L'importance des codes autocorrecteurs dans les systèmes critiques
- L'implémentation d'algorithmes mathématiques complexes en Java

5.2 Difficultés Rencontrées

1. **Positionnement des Bits** : Assurer que les bits de parité sont aux bonnes positions (puissances de 2) tout en insérant correctement les bits de données
2. **Calcul du Syndrome** : Comprendre comment les bits de parité individuels combinés donneraient la position exacte de l'erreur
3. **Gestion des Indices** : La conversion entre indices 0-based (Java) et 1-based (théorie Hamming) a nécessité une attention particulière

5.3 Limites du Programme

- **Correction Simple** : Ne peut corriger qu'une seule erreur par mot de code
- **Pas de Gestion d'Erreurs Multiples** : Si 2 erreurs ou plus se produisent, le code ne peut pas les corriger (mais peut les détecter grâce à des bits de parité supplémentaires)
- **Efficacité Réduite pour Longs Messages** : Le ratio données/code diminue avec la longueur (peut être amélioré avec des codes entrelacés)
- **Interface Basique** : Pas de visualisation étape-par-étape du processus

5.4 Perspectives d'Évolution

1. **Hamming Étendu** : Ajouter un bit de parité global pour détecter et corriger les erreurs doubles
2. **Visualisation Pédagogique** : Afficher étape-par-étape le calcul des parités et du syndrome
3. **Codes Entrelacés** : Implémenter l'entrelacement pour améliorer la détection des erreurs en rafales
4. **Codes Linéaires Généralisés** : Implémenter d'autres codes correcteurs (Reed-Solomon, BCH, LDPC)
5. **Benchmark de Performance** : Comparer l'efficacité du code avec d'autres schémas de correction
6. **Simulation de Canal Bruité** : Générer automatiquement des erreurs selon un modèle statistique

5.5 Conclusion Générale

La réalisation de ce TP sur le code de Hamming a été extrêmement enrichissante. Elle a démontré comment la théorie mathématique de la correction d'erreurs peut se traduire en algorithmes pratiques et efficaces. Contrairement au CRC qui détecte les erreurs, Hamming les corrige automatiquement, ce qui en fait un outil indispensable dans les systèmes critiques (transmissions spatiales, mémoires RAM, communications réseau).

Cette implémentation solide jette les bases pour explorer des codes plus avancés et des applications dans des domaines comme les télécommunications, l'archivage de données et l'informatique quantique.